

AcademiX: LMS multi-tenant universitario

Entrega 1

IIC3143 Desarrollo de Software · Equipo AcademiX · 5 de septiembre de 2026

1. Razón y justificación de la solución

Introducción y contexto. Las universidades usan plataformas LMS para centralizar cursos, material académico, evaluaciones, entregas y comunicación. En la práctica, buena parte del flujo académico termina repartida entre el LMS institucional, planillas de cálculo, correos, calendarios externos y carpetas de archivos. La consecuencia es una experiencia fragmentada para los tres actores que sostienen un curso semestral: estudiantes, cuerpo docente y administración académica.

Problema y motivación. El problema no es la falta de funcionalidades en los LMS existentes, sino la baja integración del flujo crítico de un curso: *curso* → *módulos* → *material* → *evaluaciones* → *entregas* → *corrección* → *publicación de notas* → *libro de notas*. Se observan cuatro síntomas: (i) el estudiante no distingue con rapidez qué estudiar, qué entregar y qué notas ya fueron publicadas; (ii) las notas y ponderaciones se gestionan fuera de la plataforma; (iii) el material queda como archivo pasivo, sin vínculo con la evaluación que lo usa; y (iv) las instituciones necesitan aislar datos y escalar por universidad. Este diagnóstico surge principalmente de la experiencia del equipo utilizando Canvas. En esta etapa no se realizaron entrevistas ni mediciones formales, por lo que se trata de una observación inicial y se reconoce esa limitación. La propuesta busca, por tanto, que las tareas principales de un curso puedan realizarse de forma continua dentro de una misma plataforma, reduciendo la dependencia de planillas, carpetas, calendarios y otras herramientas externas.

Valor de negocio. Para la institución, el valor está en reducir la dependencia de planillas externas, dejar evidencia y trazabilidad de entregas y calificaciones, mantener control sobre los datos de cada universidad y operar varias instituciones sin mezclar información. Para el cuerpo docente, en configurar y calcular calificaciones dentro de la plataforma y controlar cuándo se publican. Para el estudiante, en claridad sobre sus calificaciones y su avance en el curso. Se busca una integración del flujo académico crítico, más que un posicionarse en paridad funcional con plataformas como Canvas o Moodle.

2. Interesados y usuarios

Resumen de interesados (afectados por el resultado, no necesariamente usuarios diarios):

Interesado	Interés principal	Qué espera de AcademiX
Universidad / Dirección de docencia	Continuidad operacional, seguridad y control institucional de los datos.	Operar sus cursos sin fuga de información entre instituciones.
Departamento académico	Administrar periodos, cursos, secciones y equipos docentes.	Coordinar la oferta académica y los permisos del semestre.
Cuerpo docente	Organizar material, evaluaciones, corrección y calificaciones.	Un único lugar para el ciclo completo del curso.
Equipo técnico del proyecto	Arquitectura mantenible, desplegable y observable en un semestre.	Un alcance acotado que se pueda construir y demostrar.

Resumen de usuarios (roles que operan el sistema; un usuario puede tener distintos roles según tenant y curso):

Usuario	Rol funcional	Acciones clave
Administrador de tenant	Gestión institucional dentro de una universidad.	Usuarios, periodos académicos, cursos, secciones e inscripciones.
Docente	Dueño del curso.	Configura curso, módulos, material, evaluaciones y ponderaciones; corrige y libera notas.
Ayudante docente	Apoyo delegado con permisos acotados.	Corrección, comentarios, material, anuncios y correcciones según permiso.
Estudiante	Consumidor del flujo académico.	Revisa cursos y pendientes, identifica a su cuerpo docente, consume material, entrega y consulta notas liberadas.

3. Necesidades a resolver

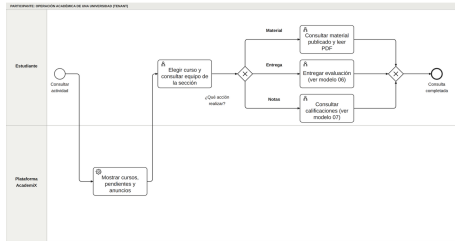
Descripción del producto. AcademiX es un LMS multi-tenant para instituciones de educación superior. Cada universidad opera como un tenant aislado, con sus propios usuarios, cursos, archivos, evaluaciones y notas. El producto organiza el semestre alrededor del curso: el material vive en módulos publicables u ocultables, las evaluaciones tienen fecha, puntaje y ponderación, las entregas dejan evidencia con estado, la corrección admite nota y comentarios, el sistema calcula el promedio según las ponderaciones configuradas y la publicación de notas es una acción explícita del docente.

Necesidad	Rasgo del producto	Beneficio
Saber qué hacer ahora	Dashboard con cursos, evaluaciones próximas y anuncios recientes.	Menos plazos perdidos y menos búsqueda manual.
Saber quién responde por el curso	Cuerpo docente de la sección visible con nombre y rol.	El estudiante sabe a quién dirigirse, sin exponer participantes ajenos al curso.
Centralizar el material	Módulos con estado publicado u oculto, archivos y enlaces.	Material vinculado al curso y visible sólo cuando el docente lo decide.
Gestionar evaluaciones	Evaluaciones con fecha, puntaje y ponderación; entregas con estado.	Evidencia del envío y control del ciclo completo.
Calificar con transparencia	Cálculo de promedios según ponderaciones, liberación manual y libro de notas.	El docente deja de depender de planillas; el estudiante ve sólo lo liberado y sabe si su promedio es parcial.
Aislar instituciones	Resolución de tenant por request, base de datos por tenant y roles contextuales.	Menor riesgo de acceso cruzado y escalamiento por universidad.

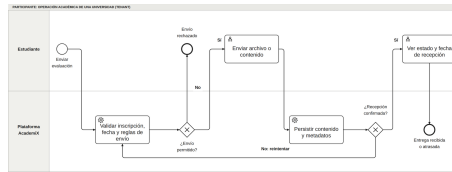
Fundaciones y extensiones. Se plantean cinco capacidades prioritarias o *must have*: representar cursos y secciones; asociar estudiantes, docentes y ayudantes; mostrar al estudiante su cuerpo docente con nombre y rol; crear y administrar evaluaciones; y registrar, calcular, publicar y visualizar calificaciones. Además, se plantean extensiones *nice to have* como: video como material, resumen de material mediante IA, chat de sección, chat de grupo, calendario en el dashboard y alertas internas. Estas extensiones sólo se incorporarán después de validar las fundaciones y si el avance permite absorber su complejidad.

Suposiciones. (a) El tenant corresponde a una universidad y el curso a un ramo semestral concreto. (b) Todo acceso ocurre con usuario autenticado. (c) El idioma principal de la interfaz es el español. (d) El MVP no reemplaza todas las funcionalidades de un LMS completo. (e) El modelo inicial considera entregas individuales. (f) Supuestos de escala que orientan la arquitectura sin exigir demostrarla en producción: hasta 5 universidades de hasta 30.000 estudiantes cada una (150.000 potenciales), con *spikes* de aproximadamente 1.000 estudiantes concurrentes en ventanas de evaluación, entrega o publicación de notas.

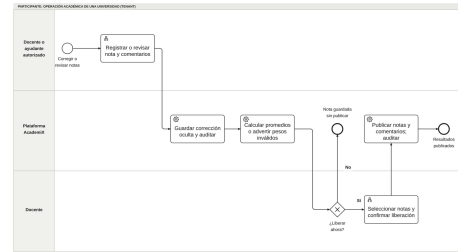
Modelo de proceso. Los diagramas representan tres momentos del flujo académico: la consulta de actividad del estudiante, el envío de una evaluación y la corrección con publicación de resultados. En cada proceso se distinguen las acciones de los usuarios y las tareas de la plataforma, junto con las decisiones que condicionan su avance. Se destacan dos controles: una entrega sólo se confirma después de persistir su contenido y metadatos, y una calificación permanece oculta hasta que el docente decide liberarla.



Actividad del estudiante



Entrega del estudiante



Corrección y publicación

Figura 1. Modelos de Proceso: consulta de cursos, material y calificaciones; validación y recepción de entregas con posibilidad de reintento; y registro de correcciones, cálculo de promedios y liberación explícita de notas.

4. Riesgos

Tipo	Riesgo	Impacto	Mitigación
Desarrollo	Sobrealcance funcional	No llegar a un MVP demostrable al cierre del semestre.	Fundaciones primero; las extensiones <i>nice to have</i> sólo se activan una vez validadas.
Desarrollo	Complejidad de multi-tenancy	Errores de aislamiento de datos o de permisos.	Registry central, estructura de datos idéntica en cada tenant y pruebas explícitas de acceso cruzado.
Desarrollo	Grupos de estudiantes	Cambiar el modelo de entregas a mitad de camino.	Mantener entregas individuales hasta especificar membresías, permisos y autoría grupal.
Desarrollo	Carga en ventanas críticas	Fallas justo en cierres de entrega y publicación de notas.	Primero consultas eficientes y backend stateless replicable; balanceo, pool, caché y workers entran cuando el plan técnico los justifique.
Desarrollo	Crecimiento de archivos	Saturación de la base de datos y costo de almacenamiento.	Binarios en object storage; en la base sólo metadatos y referencias.

Tipo	Riesgo	Impacto	Mitigación
Negocio	Agotamiento del crédito cloud	Perder el ambiente público antes del cierre del proyecto.	Crédito acotado al periodo del proyecto, servicios administrados con escala a cero y límites declarados como supuesto.
Negocio	Competencia con LMS establecidos	Diferencial poco claro frente a Canvas o Moodle.	Competir por integración del flujo crítico y transparencia de calificaciones, no por cantidad de funcionalidades.
Negocio	Baja adopción del flujo	El equipo docente y los estudiantes vuelven a planillas y carpetas externas.	Cada vista nace de una historia con criterios de aceptación y se valida en maquetas antes de programar.
Negocio	IA prematura	Costo variable y distracción del core académico.	Mantener la IA posterior al core, acotada a material publicado o al documento abierto, con citas verificables.

5. Detalle de la solución propuesta

Requisitos funcionales, trazabilidad y beneficio. El alcance se ordena por bloques; cada bloque referencia requisitos e historias con su prioridad MoSCoW. Los RFO son requisitos funcionales opcionales para las funcionalidades *nice to have*, condicionados al avance.

Bloque	RF	Historias (prioridad)	Beneficio para cliente y usuario
Tenant, permisos y cuerpo docente	RF1–RF5, RF21	US5–US7, US30 (must), US8 (should)	Cada universidad opera aislada, los roles se asignan por tenant y por curso, y el estudiante identifica a su equipo docente.
Contenido del curso	RF6–RF8	US9, US11 (must), US10, US12 (should)	El material queda vinculado al curso y visible sólo cuando el docente publica.
Evaluaciones y entregas	RF9–RF12	US13–US15 (must), US16 (should)	Fecha, puntaje y evidencia de envío con estado explícito para ambos lados.
Cálculo, corrección y notas	RF13–RF16, RF19, RF22	US17–US20 (must), US21 (should)	Promedios calculados según ponderaciones, comentarios registrados y publicación manual; el estudiante sabe cuándo su promedio es parcial.
Dashboard y anuncios	RF18	US1, US2 (must), US3, US4, US23, US24 (should)	Vista única de pendientes y comunicación del curso.
Extensiones del core	RF20 (should), RF17, RFO5 (could)	US26 (should), US22, US25 (could)	Lector de PDF en plataforma; corrección y calendario interno si el avance lo permite.
Extensiones <i>nice to have</i>	RFO1–RFO4, RFO6 (could), RFO7 (fuera)	US27–US29, US31–US36 (could), US37 (fuera)	Video, resumen mediante IA, chats y alertas internas; las alertas por correo quedan como evolución posterior.

Los RF1 (autenticación), RF6 (configuración general del curso) y RF19 (registro de cambios) son habilitadores transversales para otros requerimientos y por lo tanto no cuentan con historia propia. Se validan a través de las historias de los bloques que los usan.

Restricciones. (i) Ventana efectiva estimada de 10 a 12 semanas dentro de un semestre académico. (ii) Despliegue público con costo controlado. (iii) MVP acotado al core académico. (iv) Google Calendar, Outlook y quizzes en vivo quedan fuera del primer MVP. (v) La IA queda como extensión experimental posterior al core. (vi) El modelo inicial considera entregas individuales: grupos, chats grupales y entregas grupales exigen especificación previa. (vii) Las extensiones *nice to have* sólo se activan después de validar las fundaciones.

Requisitos no funcionales como atributos de calidad. Cada RNF se declara con la acción de diseño que lo sostiene y la evidencia con que se verificará. En lugar de metas numéricas prematuras, se fijan escenarios de calidad que describen la reacción esperada frente a fallas probables.

Atributo (RNF)	Acción de diseño / mitigación	Evidencia verificable
Aislamiento y seguridad (RNF3, RNF4)	Tenant resuelto en cada request contra un registry central; base de datos por tenant; autorización por TenantMembership y CourseMembership.	Escenario de acceso cruzado: la operación se bloquea cuando tenant, curso o sección no coinciden, con pruebas automatizadas.
Auditabilidad (RNF5)	Registrar actor, fecha, acción, valor anterior y valor nuevo en cambios de notas, ponderaciones, entregas y liberaciones.	Reconstruir el historial completo de una nota desde la auditoría.
Integridad de la evidencia académica	Confirmar la recepción de una entrega sólo cuando archivo y metadatos quedan persistidos; ante falla, error recuperable y reintento.	Prueba de subida fallida que no deja entrega ni nota inválida.

Atributo (RNF)	Acción de diseño / mitigación	Evidencia verificable
Disponibilidad (RNF1)	Despliegue reproducible en servicios administrados, logs de error y capacidad de reinicio o redespiegue; balanceo cuando existan varias réplicas.	Escenario de caída parcial de instancia y monitoreo durante las ventanas críticas del semestre.
Rendimiento (RNF6)	Respuesta estable en carga normal y degradación controlada en ventanas críticas: optimizar consultas, acotar bloques secundarios, agregar índices y medir endpoints <i>antes</i> de introducir caché.	Medición de los endpoints de dashboard y curso como condición previa a habilitar caché.
Escalabilidad (RNF2)	Frontend y backend stateless replicables; el tenant es el shard desde el inicio; réplica de lectura por tenant cuando ese tenant la justifique.	Supuesto de carga documentado (<i>spike</i> cercano a 1.000 concurrentes) usado como caso de diseño del flujo de entrega.
Persistencia de archivos (RNF8)	Binarios en object storage; en PostgreSQL sólo metadatos y <code>storage_key</code> .	Recursos y entregas recuperables desde el object storage con su metadato.
Recuperabilidad y observabilidad (RNF9, RNF10)	Backup y restore por tenant; logs y métricas básicas de error y carga.	Restore de un tenant sin afectar a los demás; trazas disponibles ante error operacional.
Mantenibilidad (RNF7)	Monolito modular por dominio antes de considerar microservicios.	Módulos separados por dominio y decisiones registradas como ADR.

Tecnologías y justificación. Stack: Next.js como cliente web que consume la API (vistas por rol, dashboard y curso), NestJS como monolito modular con límites de dominio explícitos, PostgreSQL para los datos académicos transaccionales (notas, ponderaciones y estados exigen restricciones y transacciones) y Docker para un entorno reproducible entre integrantes.

Proveedor: Google Cloud Platform, por su crédito gratuito de USD 300 por tres meses, que cubre el desarrollo y la demo final: Cloud Run para frontend y backend como contenedores stateless, Cloud SQL para las bases por tenant y el registry, Cloud Storage para binarios y Cloud Load Balancing cuando se requiera. Esta elección permite a la arquitectura no estar ligada a un proveedor y en principio los componentes se pueden portar de un proveedor a otro.

Condicional: balanceador o API gateway al desplegar más de una réplica, y pool de conexiones (PgBouncer) cuando la cantidad de tenants o la concurrencia presione los límites de conexiones.

En veremos: Redis y workers entran juntos — Redis es la cola de los workers — sólo al implementar las historias que los justifiquen: procesamiento de archivos (US11, US14), recálculo de promedios al cambiar ponderaciones (US17) y despacho de notificaciones (US35, RFO6). Hasta entonces el procesamiento es sincrónico.

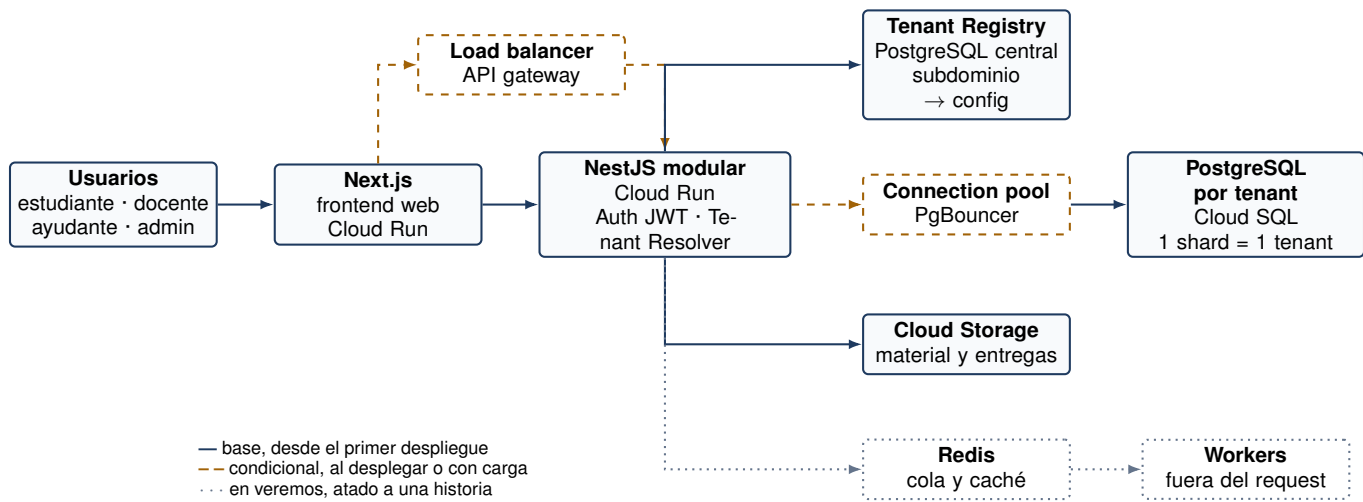
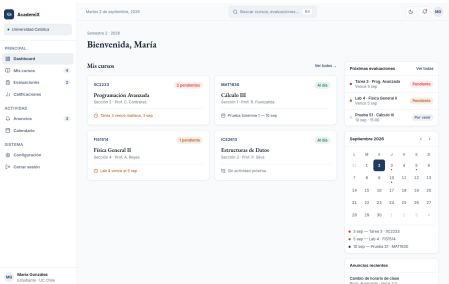
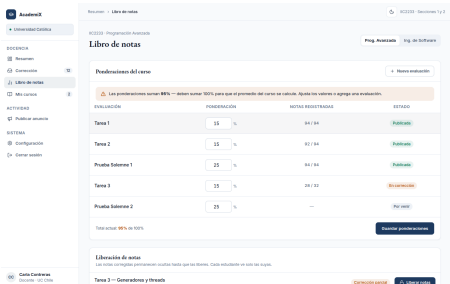


Figura 2. Arquitectura candidata. La ruta base es cliente, frontend, backend modular, registry, base por tenant y object storage; el resto está diseñado pero no comprometido para el primer despliegue.

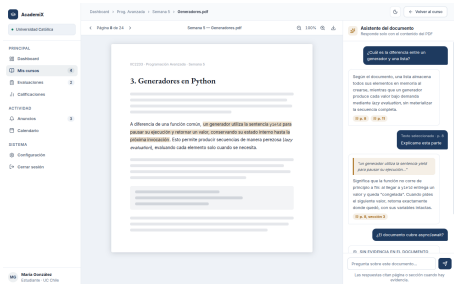
Primeras maquetas. Las maquetas son wireframes navegables en HTML estático; cada vista declara las historias que valida y no tiene backend.



Dashboard estudiantil (US1–US3)



Libro de notas docente (US17, US18)



Lector PDF e IA (US26–US29)

Figura 3. Maquetas de referencia: core del estudiante, control docente de calificaciones y extensión condicionada.

Decisiones abiertas. Se declaran para no presentar como cerrado lo que aún no está decidido:

- **Umbral de rendimiento.** No se comprometen cifras: se fijan midiendo los endpoints de dashboard y curso, y esa medición decide si entra caché.
- **Activación de Redis y workers.** Se resuelve al especificar la historia que los justifique; hasta entonces el procesamiento es sincrónico.
- **Modelo de grupos de estudiantes.** Habilitar chat de grupo o entregas grupales exige definir grupos, membresías y autoría; sin eso, el modelo se mantiene individual.
- **Extensiones *nice to have* que entran.** Depende del avance real al cerrar las fundaciones; no se compromete el conjunto completo.
- **Modelo y costo del asistente de IA.** Sin definir, coherente con dejarlo posterior al core.

6. Plan de trabajo

El roadmap enumera 12 iteraciones semanales dentro de una ventana efectiva de 10 a 12 semanas; las iteraciones 11 y 12 operan como colchón de cierre y pueden comprimirse.

Sem.	Actividades técnicas (RF/RNF)	Actividades no técnicas	Que se espera completar
1–2	Setup del monorepo, base Next.js y NestJS, autenticación y resolución de tenant (RF1, RF2, RNF3, RNF4).	Visión, modelo de dominio, arquitectura, maquetas y backlog priorizado.	Usuario autenticado y tenant resuelto en cada request.
3–4	Periodos, cursos, secciones, inscripciones, cuerpo docente visible, módulos, material y object storage (RF3–RF8, RF20, RF21, RNF8).	Validación de maquetas de curso y refinamiento del backlog.	Un curso real con secciones, participantes y material publicado.
5–7	Evaluaciones y pendientes, entregas, corrección, comentarios y liberación manual (RF9–RF13, RF16).	Criterios de aceptación por historia y revisión cruzada entre pares.	Ciclo evaluación–entrega–corrección sin publicación automática.
8–9	Libro de notas, cálculo de promedios, auditoría, dashboard y anuncios (RF14, RF15, RF18, RF19, RF22, RNF5).	Demo intermedia y decisión de qué extensiones se activan.	Docente y estudiante ven la misma verdad sobre calificaciones.
10–11	Despliegue público, observabilidad, decisiones de rendimiento, hardening de permisos y pruebas de flujos críticos (RNF1–RNF4, RNF6, RNF9, RNF10).	Coordinación de infraestructura y ADR de las decisiones abiertas.	Ambiente público operativo con permisos verificados.
12	Cierre de deuda crítica y ajustes finales de UX.	Demo final, documentación y presentación.	Entrega final demostrable y caso de negocio validado.

7. Glosario y anexos

Tenant: universidad aislada dentro de la plataforma.

Tenant registry: base central que resuelve el subdominio a la configuración del tenant.

Shard: partición de datos; aquí un shard equivale a un tenant desde el inicio.

Curso: ramo semestral concreto de un periodo académico.

Sección: grupo de estudiantes dentro de un curso.

Módulo: bloque de contenido; semana, tema o unidad.

Recurso: material publicado en un módulo (PDF, PPT, enlace).

Evaluación: actividad con entrega, corrección, nota y ponderación.

Entrega: envío de un estudiante para una evaluación.

Liberación de notas: acción del docente que hace visible una nota.

Libro de notas: vista consolidada de evaluaciones, ponderaciones y promedio.

Recorrección: solicitud de revisión de una calificación publicada.

Rol contextual: rol que depende del tenant y del curso, no del usuario global.

Escenario de calidad: falla probable descrita junto a la reacción esperada del sistema.

Object storage: almacenamiento de binarios fuera de la base relacional.

RFO: requisito funcional opcional, condicionado al avance del proyecto.

Anexo A. Trazabilidad RF → historias de usuario. RF1: habilitador. RF2: US5, US8, US12, US30. RF3: US5. RF4: US5, US6. RF5: US7, US8. RF6: habilitador. RF7: US9, US10. RF8: US11, US12. RF9: US13, US17. RF10: US2, US16. RF11: US14. RF12: US15. RF13: US18. RF14: US19, US20. RF15: US17, US19, US20. RF16: US15, US21. RF17: US22. RF18: US1–US4, US23, US24. RF19: habilitador (soporta RNF5). RF20: US26. RF21: US30. RF22: US17, US20. RFO1: US31. RFO2: US32. RFO3: US33. RFO4: US34. RFO5: US25. RFO6: US35, US36. RFO7: US37 (fuera del alcance inicial).